

MagicCode Super-Resolution Software Release Notes

Document Version: v2.2.0. Release Date: September 30, 2026.

1. Summary

MagicCode Super-Resolution Software v2.2.0 publishes four public `alg_mode_e` values (spatial Speed/Balanced and temporal Speed/Balanced) through the `nscaler` API (`mc_nscaler_enable` / `mc_nscaler_control` / `mc_nscaler_disable` / `mc_nscaler_version`), adds the Linux x86_64 platform library, and documents the temporal contract on Android Vulkan/OpenGL ES, Apple Metal, Windows Vulkan / Direct3D 11 / OpenGL 4.3, and Linux Vulkan (source build).

2. Key Changes

v2.2.0 (September 24, 2026; documentation refresh September 30, 2026):

- Public version string is `v2.2.0` (`mc_nscaler_version`).
- Public API is `mc_nscaler_enable` / `mc_nscaler_control` / `mc_nscaler_disable` / `mc_nscaler_version`. It replaces the v2.0.0 `MC_Init` / `MC_Process` / `MC_Uninit` / `MC_Control` entry and the v2.1.0 `MC_Enable` / `MC_Disable` helper; `mc_enable.h` is removed.
- `ctrl_param_t` replaces `input_param_t` (no `struct_size`/width/height; the image size moves to `mc_nscaler_input_frame_t` / `mc_nscaler_output_frame_t`, which replace `magic_frame_t`).
- Fixed a Vulkan Balanced 1.5x SIGSEGV (FSR1 timestamp query pool creation).
- Linux x86_64 platform library: `lib/linux/libmagic_sr.a` (CPU spatial, AVX2/FMA; rebuild via `tools/build_linux_lib.sh`). The archive carries no GPU backend objects; Linux Vulkan temporal builds from source and is validated headless (lavapipe) with output matching the Windows NVIDIA reference.
- Spatial (1,2) open band: BALANCED routes 2dir LUT then lanczos2HV; SPEED routes 2dir LUT then bilinear. The `lut_boost` last convolution treats out-of-image taps as zero.

v2.0.0 (September 2, 2026) key changes:

- Public version string is `v2.0.0` (`MC_GetVersion`).
- `alg_mode_e`: `SPATIAL_SPEED_MODE=0`, `SPATIAL_BALANCED_MODE=1`, `TEMPORAL_SPEED_MODE=2`, `TEMPORAL_BALANCED_MODE=3`.
- Temporal scaler range is (1, 8]. Spatial remains [1.0, 8.0] subject to backend and model.
- Formal temporal entry is `MC_Init` / `MC_Process` / `MC_Uninit` with `magic_frame_t` + `temporal_frame_t` (color/output, depth, motion, jitter, `frame_index/reset_history`, camera, `gpu_context`, `command-buffer`).
- `MC_Enable` still processes one color texture and is not a complete temporal entry.

- Windows product archives: lib/windows/libmagic_sr.lib and libmagic_enable_sr.lib. Enable does not accept MAGIC_BACKEND_D3D11.
- Both temporal modes default to deriving a missing reactive mask (and approximate transparency on the Metal/Vulkan/GLES Speed path) and to depth+MV history rejection. Explicit masks win per channel.

3. Compatibility

Platform	GPU backends	Temporal
Android	Vulkan, OpenGLES	Temporal GPU yes; CPU temporal no
iOS / iPadOS / macOS	Metal	Temporal GPU yes; CPU temporal no
Windows	Vulkan, Direct3D 11, OpenGL 4.3	Temporal GPU yes; CPU temporal no
Linux x86_64	Vulkan (source build)	Shipped archive is CPU spatial (AVX2), no GPU objects; Linux Vulkan temporal validated from source (headless); CPU temporal no

- Operating systems: Windows 10 64-bit and above, Android 8.0 and above, iOS 13.0 and above, iPadOS 13.0 and above, macOS 12.0 and above, and Linux x86_64 (glibc; CPU spatial archive, Vulkan temporal from source).
- Input resolution: 64×64 to 4032×4032.
- CPU SIMD (spatial CPU paths): AVX2 on x86_64, NEON on ARM64. CPU temporal is unavailable.

Temporal Super-Resolution

Temporal super-resolution runs through the nscaler session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable. v2.2.0 replaced the v2.0.0 MC_Init / MC_Process / MC_Uninit entry and removed the v2.1.0 MC_Enable helper (mc_enable.h is deleted). A NULL *handle on mc_nscaler_enable creates a session with defaults (2x SPATIAL_SPEED) and processes the first frame.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].

- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGL ES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGL ES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2x has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- mc_nscaler_input_frame_t / mc_nscaler_output_frame_t: caller-owned color and output GPU resources.
- mc_nscaler_input_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at create (mc_nscaler_control SET_PARAM, or the first mc_nscaler_enable): Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires mc_nscaler_input_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on ctrl_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Migration Notes

From v2.0.0 / v2.1.0 to v2.2.0:

- Rename MC_Init to mc_nscaler_control(MC_NSCALER_CMD_SET_PARAM) create (or NULL-handle mc_nscaler_enable), MC_Process to mc_nscaler_enable, MC_Uninit to mc_nscaler_disable, MC_GetVersion to mc_nscaler_version.
- Replace input_param_t with ctrl_param_t (no struct_size/width/height) and magic_frame_t with mc_nscaler_input_frame_t / mc_nscaler_output_frame_t.
- MC_Enable and mc_enable.h are removed; default-create covers the one-texture spatial case.

v2.0.0 migration notes (historical):

- From v1.2.x: keep numeric 0/1 for spatial modes (now named SPATIAL_*). Add 2/3 only when integrating temporal.
- Replace any MC_Process(handle, image_in, image_out) usage with MC_Process(handle, magic_frame_t*).
- Set input_param_t.struct_size and temporal_frame_t.struct_size.
- Do not treat Enable temporal enums as a full temporal integration.
- Review Privacy Policy and Data Reporting Notice before distributing applications that include the SDK or plugins. Those legal documents are unchanged by this technical release.

5. Known Limitations

- Not every backend, scale, or device class is device-run. Distinguish API support from verified configurations.
- Derived masks are approximations. Omitting transparency is not equivalent to a semantic transparency mask.
- Vulkan temporal requires a recording command buffer and explicit layouts; the library does not begin, end, or submit it.
- Linux: the shipped archive (lib/linux/libmagic_sr.a) is CPU spatial only; Linux Vulkan temporal is a source build, not part of the archive.
- Performance depends on device, driver, backend, resolution, scale, and motion quality.

6. Validation Snapshot

API contracts are defined by interface/mc_interface.h and the GPU host implementations.

Compile/static coverage includes several temporal rates; that is not a device-run of every rate. Linux validation: mc_interface_api_test 118/118 on the shipped archive; headless Linux Vulkan temporal itest 10/10 (lavapipe) with output spans matching the Windows NVIDIA reference.